

TARS: A Theory-of-Mind Agent for Personalized In-IDE Code Comprehension

Leopoldo Todisco*, Antonio Della Porta*, Stefano Lambiase[†], and Fabio Palomba*

*Software Engineering (SeSa) Lab, University of Salerno, Italy

[†]Human Augmentation and Collaboration (HAC) Group, Aalborg University, Denmark

Abstract—Code comprehension is one of the most time-consuming tasks in software engineering, yet most LLM-based assistants produce explanations that ignore who is asking and force developers into a disruptive copy-paste workflow. We present TARS, an LLM-powered agent integrated into Visual Studio Code that supports program comprehension through autonomous explanations anchored directly to the code under analysis. Built around a lightweight Theory of Mind paradigm, TARS profiles a developer’s expertise, role, and stylistic preferences, then adapts the depth and tone of its explanations accordingly, grounding them in project documentation via Retrieval-Augmented Generation. To evaluate TARS, we conducted a controlled experiment with 18 participants on non-trivial Java snippets. Participants using TARS completed tasks 26% faster, reported lower cognitive load, and found the explanations meaningfully adapted to their profiles.

Tool repository: <https://github.com/leotodisco/tars>

Tool video: <https://www.youtube.com/watch?v=RBvMKisWDLM>

Index Terms—Code Comprehension, Large Language Models, Cognitive Theories

I. INTRODUCTION

Code comprehension is a fundamental activity in software engineering, underlying virtually every development task [1], [2]. Its importance is most pronounced during software maintenance and evolution, which dominate the software life cycle and consume the majority of its total cost [1], since before a developer can fix a bug, extend a feature, or refactor a module, they must first understand code they rarely authored themselves. The ability to rapidly build an accurate mental model of an unfamiliar codebase is therefore a critical, yet cognitively demanding, skill [3], [4], and developers spend a substantial portion of their time not writing code, but searching for, reading, and making sense of code written by others [5], [6]. Among the aids that support this process, source code comments and documentation have consistently been identified as the most critical [7], [8]. Yet a well-known paradox persists: despite their recognized importance, comments in practice are frequently incomplete, outdated, or incorrect [9], leaving insufficiently documented code as a leading source of comprehension difficulties [10].

In recent years, researchers have begun investigating how Large Language Models (LLMs) can address this longstanding documentation dilemma. Pioneering tools such as GILT [11] and IVIE [12] have demonstrated that LLMs can shift the developer workflow from static information *retrieval* to dynamic information *generation*, producing on-demand, context-aware explanations tailored to the code at hand. While

these results are encouraging, existing tools remain limited in several respects. They typically target isolated code snippets and operate reactively, responding only to explicit user queries rather than proactively assisting comprehension. More fundamentally, they generate “one-size-fits-all” explanations that overlook the individual developer’s background, skill set, and immediate goals. By failing to account for user expertise, current tools create a cognitive mismatch, either overwhelming less-experienced developers with low-level jargon or frustrating seasoned engineers with redundant, elementary details.

The difficulty of these mismatches is best understood through cognitive theories of comprehension, which cast it as an active, hypothesis-driven process: developers build understanding either bottom-up, by chunking statements into higher-level abstractions [13], or top-down, by refining expectations about the code’s purpose [14]. Which strategy a developer follows and what they need to advance it depend on their prior knowledge or expertise, so an explanation that ignores this risks disrupting rather than supporting comprehension.

This is where Theory of Mind (ToM) becomes relevant. Originating in cognitive science, ToM describes the human ability to attribute mental states to others and to reason about what they believe, intend, and do not yet know [15]. It is what allows a senior developer to tailor an explanation to a colleague rather than reciting the same account to everyone. Recent work shows that this capacity can be partially elicited in LLMs through prompting [16]–[19], enabling assistants to maintain an implicit model of the developer’s evolving understanding and shape explanations to fill gaps and avoiding cognitive overload.

To this end, we introduce TARS, an LLM-powered agent that enhances code comprehension through autonomous, context-sensitive assistance integrated directly into the developer’s IDE. Unlike prior tools that treat LLMs as passive question-answering engines, TARS leverages an explicit perspective-taking mechanism to condition its explanations on the developer’s expertise, role, and stylistic preferences, adapting their depth, tone, and granularity to each user. A Retrieval-Augmented Generation component further grounds explanations in project-specific documentation. By embedding the agent within the editor, TARS minimizes the cognitive overhead of context switching and anchors its explanations directly to the code under analysis.

We evaluate TARS through a controlled within-subjects experiment in which 18 participants completed code compre-

hension tasks on non-trivial Java snippets under both assisted and unassisted conditions. Results show that TARS reduces task completion time by approximately 26%, is perceived as highly useful and easy to use, imposes significantly lower cognitive workload than the manual condition, and produces explanations that participants find meaningfully adapted to their individual profile.

II. BACKGROUND AND RELATED WORK

LLMs offers a structural response to the documentation dilemma, replacing static, human-authored documentation with dynamic, on-demand explanation [11]. Where the traditional workflow cast the developer as an archaeologist excavating API references and forums for pre-existing answers, the emerging paradigm recasts them as a collaborator who prompts an LLM for a bespoke explanation of code in context. GILT [11] embodies this shift as a VS Code plugin that supports understanding through a chatbot interface, while IVIE [12] pushes it further by *anchoring* explanations directly alongside the code they describe, mitigating the split-attention effect of toggling between the editor and a separate chat window. These tools establish two principles we adopt in TARS, namely that explanations should be situated within the editor, yet they share a key limitation: the explanation is the same regardless of who requests it.

Theory of Mind offers a principled way to address this. Recent work shows ToM can be elicited in LLMs through prompting [16], [17], and Richards and Wessel [20] brought these ideas into software engineering with TOMMY, a conversational assistant that infers a user’s mental state from dialogue history to personalize explanations. TARS builds on this line of work but differs in two respects: it conditions explanations on an explicit developer profile rather than inferring state implicitly from conversation, and it delivers them anchored in the IDE rather than through a separate conversational channel.

III. TARS

A. What is TARS

TARS is an LLM-powered assistant integrated as a Visual Studio Code (VS Code) extension that supports program comprehension directly within the developer’s IDE. It is designed around two ideas: (1) explanations should be *anchored* in the editor rather than delivered through a separate window, and (2) they should be *personalized* for the single developer rather than identical for everyone.

To this end, TARS is built on a lightweight Theory of Mind paradigm. It models the developer’s cognitive state, capturing their expertise, role, and stylistic preferences, and conditions its explanations on this profile to adjust their granularity, tone, and technical depth. A junior developer and a senior architect are querying the same code, therefore, receive explanations pitched at different levels. To keep its reasoning grounded in the software under analysis, TARS further incorporates a Retrieval-Augmented Generation (RAG) component that ingests local project documentation, enriching explanations with project-specific architectural and domain knowledge.

B. How TARS works

The explanation generation process in TARS is governed by a stateful agent built with LangGraph, structured as a directed graph where nodes encapsulate discrete processing steps and edges represent conditional transitions. The graph comprises three nodes. The **Planner Node** consumes the code snippet, the user profile, and any retrieved documentation chunks, emitting a structured JSON object with the original code and a natural language explanation. The **Syntax Check Node** validates the structural integrity of this output, looping back to the planner if malformed. The **Critique Node** performs a semantic quality assessment, triggering revision when the explanation is inaccurate or insufficient, up to a configurable retry limit. Perspective-taking is operationalized in the Planner Node via a single consolidated system prompt that conditions the LLM’s output on the profile collected by the Theory of Mind Profiler: the user’s mental state, inferred through the initial questionnaire, is injected directly into the explanation prompt. While inspired by SIMTOM-Single [17], our approach differs in the timing and nature of the profile representation, as the cognitive profile is explicitly defined and structured *before* generation, thereby reducing inferential load at execution. Rather than using multi-step inference to deduce the user’s mental state at runtime [20], TARS treats expertise level, role, and stylistic preferences as static contextual directives, optimizing latency by avoiding sequential LLM calls while retaining effective profile-conditioned adaptation. When project documentation has been indexed, the most relevant retrieved chunks are appended, grounding the explanation in the project’s architecture and domain conventions.

C. How to Use TARS

Since TARS is implemented as a Visual Studio Code (VS Code) extension, the only prerequisite for installation is a working instance of VS Code. To install the tool, the user must open the VS Code Command Palette and run

```
> EXTENSIONS: INSTALL FROM VSIX
> TARS.VSIX
```

As for its use, TARS is designed to operate through three sequential steps. First, the user must execute:

```
> TARS: CONFIGURE TARS
```

which prompts the selection of a target Large Language Model (LLM) and the insertion of the corresponding API key. Then execute the command to access the ToM profiler.

```
> TARS: THEORY OF MIND PROFILER
```

This step presents the user with a structured questionnaire to capture their current mental state and domain expertise. The collected profile is then used by TARS to perform perspective taking, i.e., to tailor the generated explanations to the specific cognitive and knowledge context of the individual developer. Figure 1 shows the profiler interface as it appears within the VSCode editor. Finally, once the profiling phase is completed, the user can request source code explanations by invoking

```
> TARS: EXPLAIN CODE
```

The scope of the explanation **adapts dynamically to the user’s selection**: if a code snippet is highlighted in the editor at

Fig. 1: The Theory of Mind Profiler questionnaire presented to the user within the VSCode editor, used to capture expertise level, role, and stylistic preferences.

Fig. 2: Example of an explanation generated by TARS for a code snippet

the time of invocation, TARS produces a targeted explanation for that fragment; otherwise, the explanation covers the entire file currently open in the editor. Figure 2 shows an example of an interaction between TARS and a developer.

IV. EVALUATION OF THE TOOL

A. Study design

To assess the effectiveness of TARS, we conducted a controlled experiment with 18 participants drawn from three groups: practitioners, researchers, and students. This heterogeneity is deliberate, since TARS adapts to user profiles, its effect can only be validated against users whose expertise and background genuinely differ. The overall goal of the current work is the following:

© Goal of the work

Assess whether TARS effectively supports developers in understanding unfamiliar or legacy code, with respect to explanation quality, cognitive load, task performance, and adaptability to individual user characteristics.

To reach the goal and guide our investigation, we formulated the following research questions:

RQ₁—Does the use of TARS improve code comprehension compared to the control condition?

RQ₂—How is TARS perceived by users in terms of utility, ease of use, and cognitive load?

RQ₃—Is TARS capable of adapting its explanations to the user’s individual characteristics?

We employed a within-subjects experimental design to evaluate the impact of TARS on code comprehension. Under this design, all participants interacted with the same four selected Java code snippets, analyzing two snippets using the TARS-assisted condition and the remaining two without any AI assistance.

To mitigate issues such as learning and sequence effects, we implemented a comprehensive, two-tiered counterbalancing strategy across the participants set:

- **Condition Order (Macro-categories):** To prevent the results from being biased by participants simply becoming more familiar with the experimental procedure over time, the cohort was evenly divided into two macro-categories. The first group began their session using the TARS tool for the first two snippets, then switched to the manual condition for the remaining two. Conversely, the second group started with the manual control condition and subsequently transitioned to the TARS-assisted condition. This crossover approach ensures that fatigue or learning effects are symmetrically distributed across both conditions.
- **Snippet Assignment Combinations:** To ensure that the intrinsic difficulty of any specific code snippet did not skew the performance metrics of a particular condition, we systematically rotated the assignment of the four snippets. Given four snippets, there are six possible pairwise combinations for allocating them into two sets (e.g., assigning snippets 1 and 2 to TARS while assigning 3 and 4 to the manual condition; assigning 1 and 3 to TARS, and 2 and 4 to manual, etc.). Participants in both macro-categories were evenly distributed across these six combinatorial sequences.

TABLE I: Comparison of comprehension performance under TARS-assisted and control conditions

Measure	Condition	Mean (SD)	Median
Completion Time (s)	TARS	224.00 (116.74)	213.5
	NO TARS	304.08 (215.03)	279.5
Cosine Similarity	TARS	0.888 (0.008)	0.889
	NO TARS	0.886 (0.007)	0.887

To select the snippets that users have to understand, we selected four Java code snippets sourced from the CodeSearch-Net dataset [21], part of the CodeXGLUE benchmark [22]. Snippets were selected by filtering for a cyclomatic complexity [23] between 10 and 30 using the `lizard` library, aiming for non-trivial but manageable code. All pre-existing comments were removed to avoid bias. The LLM used to run TARS was standardized to GPT-4.1 Nano across all participants, each of whom received a dedicated API key inside the task description.

To answer **RQ₁**, we measured *task execution time* and *task correctness*. Correctness was operationalized in two complementary ways: (i) as the *cosine similarity* between the participant’s explanation and the ground-truth docstring, computed after a standard NLP pre-processing pipeline using the `intfloat/multilingual-e5-base` sentence-transformer model, and (ii) through a *manual evaluation*, in which each explanation was independently classified as *Fully Correct*, *Partially Correct*, or *Not Correct* against the oracle docstring by two of the authors, with a third author resolving any disagreements.

To answer **RQ₂**, we use a post-experiment questionnaire that combines two validated instruments: the Technology Acceptance Model (TAM) [24], covering Perceived Usefulness (4 items), Perceived Ease of Use (4 items), Behavioral Intention (1 item), and Attitude Toward Use (3 items), for a total of 12 items; and a modified NASA-TLX [25], covering Mental Demand (3 items), Physical Demand (1 item), Performance (2 items), Effort (3 items), and Frustration (3 items), for a total of 12 items, used to assess cognitive load.

To answer **RQ₃**, we extended the same questionnaire described above with a dedicated custom section of 9 Trust in Machine (TOM) items, designed to assess users’ perceptions of how well the generated explanations aligned with their individual profiles and backgrounds, covering dimensions such as the appropriateness of technical depth, relevance to the user’s stated experience level, and overall personalization quality.

V. RESULTS

We discuss the results organized by research question.

A. **RQ₁**—Does the use of TARS improve code comprehension compared to the control condition?

Neither performance metric reached conventional statistical significance, yet both point in the expected direction (Table I).

TABLE II: Self-reported measures (TARS condition only)

Construct	Mean	SD	Scale
Perceived Usefulness (PU)	4.28	0.65	1–5
Perceived Ease of Use (PE)	4.33	0.67	1–5
Attitude Toward Use (ATU)	4.19	0.70	1–5
Behavioral Intention (BI)	4.61	0.61	1–5
Trust in Machine (TOM)	3.92	0.63	1–5
Mental Demand (MD)	3.64	1.56	1–5
Frustration	3.86	1.64	1–5
Effort	3.01	1.42	1–5

For *task correctness*, cosine similarity scores were nearly identical across conditions (TARS: $M = 0.888$; control: $M = 0.886$), with an independent-samples t -test confirming no significant difference ($t(16) = -1.52$, $p = 0.18$). For *task execution time*, TARS participants were on average 80 seconds faster ($M = 224.0$ s vs. 304.08 s, a 26% reduction) and showed markedly lower variability ($SD = 116.74$ vs. 215.03). Because the control-group times violated normality (Shapiro–Wilk: $W = 0.82$, $p < .001$), a Mann–Whitney U test was applied; the result did not reach significance ($U = 494.5$, $p = 0.085$), though the p -value lies close to the $\alpha = 0.05$ threshold, suggesting a power limitation rather than the absence of a true effect. The manual evaluation corroborates this picture: under the TARS condition, explanations were more often judged *Fully Correct* (7 vs. 4) and less often *Not Correct* (2 vs. 3), while the majority in both conditions were rated *Partially Correct*, indicating that participants generally grasped the core functionality but often missed finer details captured in the oracle docstring.

B. **RQ₂**—How is TARS perceived by users in terms of utility, ease of use, and cognitive load?

All TAM constructs were rated significantly above the neutral midpoint. Perceived Usefulness ($V = 2.0$, $p < .001$), Perceived Ease of Use ($V = 0.0$, $p < .001$), and Behavioral Intention ($V = 0.0$, $p < .001$) all exceeded the neutral value, as did Attitude Toward Using ($t = 7.21$, $p < .001$). On the NASA-TLX side, Mental Demand ($t = -3.69$, $p = .0018$), Effort ($t = -5.91$, $p < .001$), and Frustration ($t = -2.94$, $p < .05$) were all significantly below the neutral midpoint, while perceived Performance was significantly above it ($V = 5.0$, $p = .0003$). Participants therefore found TARS useful and easy to operate, and associated it with a low cognitive burden and high self-reported performance.

C. **RQ₃**—Is TARS capable of adapting its explanations to the user’s individual characteristics?

The mean Theory of Mind (ToM) score, calculated as the average of the scores from all ToM-related questions, was higher than the neutral midpoint of 3. This indicates that participants perceived the explanations as adapted to their individual profiles. Qualitative feedback reinforced this finding, though some participants noted a discrepancy between their declared preference for verbosity and the actual level of

detail in the output, pointing to room for improvement in how the agent operationalizes the user profile.

VI. DISCUSSIONS

The evaluation of TARS provides valuable insights into the role of personalized, in-IDE AI assistants. While not all objective metrics were statistically significant, the convergence of quantitative trends and subjective feedback points to three practical implications.

TARS as an accelerator, not a corrector. Correctness was comparable across conditions, but TARS users were on average 26% faster with nearly halved completion-time variance. Although this trend did not reach statistical significance, its direction suggests that TARS helps developers avoid getting stuck on unfamiliar code, making the effort required for comprehension more predictable.

The primacy of developer experience. The RQ₂ results are our clearest signal: participants significantly favored the TARS workflow, reporting lower mental demand, effort, and frustration. By offloading the burden of parsing unfamiliar syntax, TARS appears to preserve developers' mental energy, suggesting its value lies as much in reducing cognitive friction as in improving accuracy.

The reality of AI personalization. Participants perceived explanations as adapted to their profile rather than generic, supporting the integration of Theory of Mind into coding assistants. Yet feedback on verbosity exposes a known limitation, in that current LLMs struggle to honor abstract constraints such as "conciseness," pointing to future work on stricter prompting or post-generation filtering.

VII. CONCLUSIONS

We presented TARS, a VS Code agent grounding autonomous, in-IDE code explanations in Theory of Mind. A within-subjects study with 18 participants showed faster comprehension, lower cognitive load, and explanations perceived as personalized. Future work will scale the evaluation to professionals on real-world codebases to test generalizability.

REFERENCES

- [1] A. Von Mayrhauser and A. M. Vans, "Program comprehension during software maintenance and evolution," *Computer*, 2002.
- [2] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, "Measuring program comprehension: A large-scale field study with professionals," *IEEE Transactions on Software Engineering*, 2017.
- [3] W. Maalej, R. Tiarks, T. Roehm, and R. Koschke, "On the comprehension of program comprehension," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 23, no. 4, pp. 1–37, 2014.
- [4] A. J. Ko, R. DeLine, and G. Venolia, "Information needs in collocated software development teams," in *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 2007, pp. 344–353.
- [5] A. J. Ko, B. A. Myers, M. J. Coblenz, and H. H. Aung, "An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks," *IEEE Transactions on software engineering*, vol. 32, no. 12, pp. 971–987, 2006.
- [6] T. D. LaToza, G. Venolia, and R. DeLine, "Maintaining mental models: a study of developer work habits," in *Proceedings of the 28th international conference on Software engineering*, 2006, pp. 492–501.
- [7] S. Stapleton, Y. Gambhir, A. LeClair, Z. Eberhart, W. Weimer, K. Leach, and Y. Huang, "A human study of comprehension and code summarization," in *Proceedings of the 28th International Conference on Program Comprehension*, 2020, pp. 2–13.
- [8] R. Brooks, "Using a behavioral theory of program comprehension in software engineering," in *Proceedings of the 3rd international conference on Software engineering*, 1978, pp. 196–201.
- [9] B. Fluri, M. Wursch, and H. C. Gall, "Do code and comments co-evolve? on the relation between source code and comment changes," in *14th Working conference on reverse engineering (WCRE 2007)*. IEEE, 2007, pp. 70–79.
- [10] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, "Measuring program comprehension: A large-scale field study with professionals," *IEEE Transactions on Software Engineering*, vol. 44, no. 10, pp. 951–976, 2017.
- [11] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [12] L. Yan, A. Hwang, Z. Wu, and A. Head, "Ivive: Lightweight anchored explanations of just-generated code," in *Proceedings of the 2024 CHI conference on human factors in computing systems*, 2024, pp. 1–15.
- [13] R. Brooks, "Towards a theory of the comprehension of computer programs," *International journal of man-machine studies*, vol. 18, no. 6, pp. 543–554, 1983.
- [14] E. Soloway and K. Ehrlich, "Empirical studies of programming knowledge," *IEEE Transactions on software engineering*, 2009.
- [15] A. I. Goldman *et al.*, *Theory of mind*. Oxford handbook of philosophy and cognitive science, 2012, vol. 1.
- [16] S. R. Moghaddam and C. J. Honey, "Boosting theory-of-mind performance in large language models via prompting," *arXiv preprint arXiv:2304.11490*, 2023.
- [17] A. Wilf, S. Lee, P. P. Liang, and L.-P. Morency, "Think twice: Perspective-taking improves large language models' theory-of-mind capabilities," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- [18] A. Della Porta, G. Voria, A. Abbate, R. Sulipano, S. Lambiase, G. Catolino, and F. Palomba, "Toward measuring prompt quality: A preliminary investigation on prompt smells," in *2026 IEEE International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C)*. IEEE, 2026, pp. 293–300.
- [19] A. Della Porta, G. Recupito, S. Lambiase, D. Di Nucci, and F. Palomba, "Unlocking code simplicity: The role of prompt patterns in managing llm code complexity," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C)*. IEEE, 2025, pp. 140–143.
- [20] J. Richards and M. Wessel, "What you need is what you get: Theory of mind for an llm-based code understanding assistant," in *2024 IEEE International Conference on Software Maintenance and Evolution (IC-SME)*. IEEE, 2024, pp. 666–671.
- [21] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.
- [22] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," *arXiv preprint arXiv:2102.04664*, 2021.
- [23] T. J. McCabe, "A complexity measure," *IEEE Transactions on software Engineering*, no. 4, pp. 308–320, 1976.
- [24] F. D. Davis *et al.*, "Technology acceptance model: Tam," *AI-Sugri, MN, Al-Aufi, AS: Information seeking behavior and technology adoption*, vol. 205, no. 219, p. 5, 1989.
- [25] S. G. Hart and L. E. Staveland, "Development of nasa-tlx (task load index): Results of empirical and theoretical research," in *Advances in psychology*. Elsevier, 1988, vol. 52, pp. 139–183.